Department of CSE (AIML), PES University
UE26CS151A -Python for Computational Problem Solving
Laboratory Week 4

1) Easy(10M) - Zigzag Case Flip

Problem Statement

Given a string `s` containing only lowercase English letters, convert it to `zigzag case`:

1. Characters at even indices (0, 2, 4, …) become uppercase.
2. Characters at odd indices (1, 3, 5, …) remain lowercase.
3. Print the resulting string.

Conditions

- Use only **input, print, variables, ord(), chr(), operators, strings, lists, and control structures**.
- Do not use user-defined functions.
- Do not use `.upper()`, `.lower()`, `.swapcase()`, or any built-in string case methods.
- Do not use dictionaries, sets, or built-in string/list functions.

Test Cases

Test Case 1

Input:

```
hello
```

Output:

```
HeLlO
```

Test Case 2

Input:

```
python
```

Output:

```
PyThOn
```

Test Case 3

Input:

```
abcdef
```

Output:

```
AbCdEf
```

Test Case 4

Input:

```
a
```

Output:

```
A
```

Test Case 5

Input:

```
programming
```

Output:

```
PrOgRaMmInG
```

Boilerplate Code

```
s = input()

result = []

# Write your code here


# Print the result
out = ""
for ch in result:
    out = out + ch
print(out)
```

Complete Solution

```
s = input()

result = []

i = 0
while i < len(s):
    ch = s[i]
```

```
    if i % 2 == 0:
        # Convert lowercase to uppercase using ord/chr
        code = ord(ch)
        if 97 <= code <= 122:
            result.append(chr(code - 32))
        else:
            result.append(ch)
    else:
        result.append(ch)
    i = i + 1

out = ""
for ch in result:
    out = out + ch
print(out)
```

2) Medium(15M) - Run-Length Encoding

Problem Statement

Given a string `s` consisting of lowercase English letters, compress it using `Run-Length Encoding (RLE)`:

4. Consecutive identical characters are replaced by the character followed by its count.
5. If a character appears only once consecutively, print just the character without a count.
6. Print the compressed string.

Conditions

- Use only **input, print, variables, str(), operators, strings, lists, and control structures**.
- Do not use user-defined functions.
- Do not use dictionaries, sets, or any built-in string/list methods other than `.append()`.

Test Cases

Test Case 1

Input:

```
aaabbcddd
```

Output:

```
a3b2cd3
```

Test Case 2

Input:

```
abcdef
```

Output:

```
abcdef
```

Test Case 3

Input:

```
aabbcc
```

Output:

```
a2b2c2
```

Test Case 4

Input:

```
zzzzz
```

Output:

```
z5
```

Test Case 5

Input:

```
aabbbccccdddddde
```

Output:

```
a2b3c4d5e
```

Boilerplate Code

```
s = input()

result = []
i = 0

# Write your code here


# Print the compressed string
out = ""
for ch in result:
    out = out + ch
print(out)
```

Complete Solution

```
s = input()

result = []
i = 0

while i < len(s):
```

```
    current = s[i]
    count = 1

    while i + count < len(s) and s[i + count] == current:
        count = count + 1

    result.append(current)

    if count > 1:
        result.append(str(count))

    i = i + count

out = ""
for ch in result:
    out = out + ch
print(out)
```

3) Difficult(25M) - Pascal's Triangle

Problem Statement

Given an integer `n`, generate and print the first `n` rows of `Pascal's Triangle`.

In Pascal's Triangle:

7. The first and last values of each row are always 1.
8. Each intermediate value is the sum of the two values directly above it from the previous row: row[j] = prev_row[j - 1] + prev_row[j].
9. Print each row on a new line with space-separated integers.

Conditions

- Use only **input, print, variables, int(), str(), len(), range(), operators, lists, and control structures**.
- Do not use user-defined functions.
- Do not use math libraries, dictionaries, sets, or built-in list methods other than `.append()`.

Test Cases

Test Case 1

Input:

```
5
```

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Test Case 2

Input:

```
1
```

Output:

```
1
```

Test Case 3

Input:

```
3
```

Output:

```
1
1 1
1 2 1
```

Test Case 4

Input:

```
6
```

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

Test Case 5

Input:

```
4
```

Output:

```
1
1 1
1 2 1
1 3 3 1
```

Boilerplate Code

```
n = int(input())

triangle = []

# Write your code here
```

```
# Print the triangle row by row
for row in triangle:
    out = ""
    for k in range(len(row)):
        if k > 0:
            out = out + " "
        out = out + str(row[k])
    print(out)
```

Complete Solution

```
n = int(input())

triangle = []

i = 0
while i < n:
    row = []
    j = 0
    while j <= i:
        if j == 0 or j == i:
            row.append(1)
        else:
            val = triangle[i - 1][j - 1] + triangle[i - 1][j]
            row.append(val)
        j = j + 1
    triangle.append(row)
    i = i + 1

for row in triangle:
    out = ""
    for k in range(len(row)):
        if k > 0:
            out = out + " "
        out = out + str(row[k])
    print(out)
```